

Séance 2 : Taxonomie des Design Patterns

2.1 Vue d'ensemble des Design Patterns

DESIGN PATTERNS (23)		
CREATIONAL (5 patterns)	STRUCTURAL (7 patterns)	BEHAVIORAL (11 patterns)
• Factory • Abstract Fac. • Builder • Prototype • Singleton	• Adapter • Bridge • Composite • Decorator • Facade • Flyweight • Proxy	• Chain of Responsibility • Command • Iterator • Mediator • Memento • Observer • State • Strategy • Template Method • Visitor • Interpreter

2.2 Patterns Créationnels (Creational)

Objectif : Gérer la création d'objets de manière flexible et réutilisable.

Principe fondamental

Séparer la création de l'utilisation. Le code client ne doit pas connaître les classes concrètes.

Les 5 patterns créationnels

Pattern	Problème résolu	Solution
Factory Method	Créer des objets sans spécifier leur classe exacte	Définir une interface pour créer un objet, laissant les sous-classes décider de la classe à instancier
Abstract Factory	Créer des familles d'objets liés sans spécifier leurs classes concrètes	Fournir une interface pour créer des familles d'objets liés
Builder	Construire un objet complexe étape par étape	Séparer la construction d'un objet complexe de sa représentation
Prototype	Créer de nouveaux objets en copiant des instances existantes	Spécifier les types d'objets à créer en utilisant un prototype prototypique
Singleton	Garantir qu'une classe n'a qu'une seule instance	Fournir un point d'accès global à une instance unique

Analogie

Imaginez une **usine de voitures** :

- **Factory** : L'usine decide quel modèle produire selon la commande
 - **Abstract Factory** : Une usine produit des voitures et des camions de la même marque
 - **Builder** : Un véhicule est assemblé pièce par pièce (moteur, carrosserie, intérieur...)
 - **Prototype** : On clone un véhicule existant pour en créer un similaire
 - **Singleton** : Il n'y a qu'un seul directeur d'usine
-

2.3 Patterns Structurels (Structural)

Objectif : Combiner des objets et des classes pour former des structures plus grandes et plus flexibles.

Principe fondamental

Comment assembler des objets et des classes en des structures plus grandes tout en gardant la flexibilité et l'efficacité.

Les 7 patterns structurels

Pattern	Problème résolu	Solution
Adapter	Incompatibilité entre deux interfaces	Convertir l'interface d'une classe en une autre interface attendue
Bridge	Abstraction et implémentation couplées	Séparer une abstraction de son implémentation pour qu'elles puissent varier indépendamment
Composite	Représentation hiérarchique d'objets	Composer des objets en structures arborescentes pour représenter des hiérarchies partie-tout
Decorator	Ajouter des responsabilités à un objet dynamiquement	Attacher des responsabilités supplémentaires à un objet dynamiquement
Facade	Interface complexe d'un sous-système	Fournir une interface unifiée vers un ensemble d'interfaces d'un sous-système
Flyweight	Coût élevé de création de nombreux petits objets	Utiliser le partage pour supporter de grands nombres d'objets à granularité fine
Proxy	Contrôle d'accès à un objet	Fournir un substitut ou un placeholder pour un autre objet pour contrôler l'accès

Analogie

Imaginez un **bâtiment** :

- **Adapter** : Un adaptateur de prise électrique (transforme une interface en une autre)
- **Bridge** : Le pont entre le design et l'implémentation
- **Composite** : Un arbre organisationnel (organigramme)
- **Decorator** : Les decoration de Noël sur un arbre (on ajoute des 装饰 sans changer l'arbre)
- **Facade** : La façade d'un bâtiment (une interface simple vers les salles internes)
- **Flyweight** : Le mobilier partagé dans un espace de coworking
- **Proxy** : Le gardien qui contrôle l'entrée au bâtiment

2.4 Patterns Comportementaux (Behavioral)

Objectif : Gérer la communication et les interactions entre objets.

Principe fondamental

Comment des objets interagissent et répartissent les responsabilités entre eux.

Les 11 patterns comportementaux

Pattern	Problème résolu	Solution
Chain of Responsibility	Passer une requête le long d'une chaîne de handlers	Détacher l'émetteur de ses récepteurs en donnant à plus d'un objet la possibilité de traiter la requête
Command	Encapsuler une requête comme un objet	Encapsuler une requête comme un objet, permettant ainsi de paramétrer les clients avec différentes requêtes
Iterator	Parcourir une collection sans exposer sa représentation interne	Fournir un moyen d'accéder aux éléments d'une collection séquentiellement sans exposer sa représentation interne
Mediator	Réduire les dépendances entre objets	Définir un objet qui encapsule comment un ensemble d'objets interagissent
Memento	Sauvegarder et restaurer l'état d'un objet	Capturer et externaliser l'état interne d'un objet sans violer l'encapsulation
Observer	Notifier automatiquement les objets intéressés	Définir une dépendance un à plusieurs entre objets afin que lorsque l'état d'un objet change, tous ses dépendants sont notifiés
State	Modifier le comportement d'un objet quand son état change	Permettre à un objet de modifier son comportement quand son état interne change
Strategy	Algorithmes interchangeables	Définir une famille d'algorithmes, les encapsuler, et les rendre interchangeables
Template Method	Différer l'exécution de certaines étapes d'un algorithme	Définir le squelette d'un algorithme dans une méthode, différant certaines étapes aux sous-classes
Visitor	Ajouter des opérations à des objets sans les modifier	Représenter une opération à effectuer sur les éléments d'une structure d'objets
Interpreter	Évaluer des expressions dans un langage	Donner un langage une représentation grammaire et définir une interprétation pour ce langage grammaire

Analogie

Imaginez un **restaurant** :

- **Chain of Responsibility** : La commande passe du serveur au cuisinier au livreur
- **Command** : La commande est écrite sur un ticket (objet)
- **Iterator** : On parcourt le menu sans connaître la structure interne
- **Mediator** : Le maître d'hôtel coordonne serveurs et cuisiniers
- **Memento** : Le serveur note la commande (état) pour pouvoir la rappeler
- **Observer** : Les clients sont notifiés quand leur plat est prêt
- **State** : Le restaurant change de comportement selon l'heure (ouvert/fermé)
- **Strategy** : Différentes méthodes de cuisson selon le plat

- **Template Method** : La recette suit un ordre fixe, mais les ingrédients changent
- **Visitor** : Un inspecteur visite chaque service du restaurant
- **Interpreter** : Le serveur interprète les commandes spéciales des clients

2.5 Comment choisir le bon Pattern ?

Questions à se poser :

1. **Problème de création** ? → Patterns Créationnels
2. **Problème de structure/organisation** ? → Patterns Structurels
3. **Problème de communication/interaction** ? → Patterns Comportementaux

Matrice de décision

Situation	Pattern recommandé
Créer des objets sans connaître leur type exact	Factory Method
Construire un objet complexe avec many options	Builder
Garantir une seule instance	Singleton
Adapter une interface incompatible	Adapter
Notifier plusieurs objets d'un changement	Observer
Varier un algorithme à运行时	Strategy
Parcourir une structure complexe	Visitor



Résumé de la séance

- **3 catégories** de patterns : Creational (5), Structural (7), Behavioral (11)
- **Creational** : Gèrent la création d'objets (Factory, Builder, Singleton...)
- **Structural** : Gèrent la composition des objets (Adapter, Facade, Decorator...)
- **Behavioral** : Gèrent les interactions entre objets (Observer, Strategy, Command...)
- Le choix du pattern dépend du **problème** à résoudre, pas du pattern lui-même



Pour aller plus loin

- Créer un tableau comparatif des 23 patterns
- Lister 3 cas d'utilisation concrets pour chaque catégorie
- Explorer les relations entre patterns (ex: Factory peut utiliser Singleton)
- Le pattern [Proxy](#) utilisé en JavaScript
- Le pattern [Iterator](#) utilisé en JavaScript